

Explain Josephus Logic

Sure. Here is the **same** explanation, but more precise and focused on the actual logic.

LeetCode 1823 — Find the Winner of the Circular Game

Approach

We use the **Josephus Problem** recurrence.

We maintain the winner's **0-based index**.

Start with only 1 person:

```
int ans = 0;
```

Then gradually increase the number of people:

```
for (int i = 2; i <= n; i++) {
    ans = (ans + k) % i;
}
```

At the end, `ans` is a **0-based index**, while the people are numbered from `1`, so:

```
return ans + 1;
```

Code

```
class Solution {
public:
    int findTheWinner(int n, int k) {
        int ans = 0;

        for (int i = 2; i <= n; i++) {
            ans = (ans + k) % i;
        }

        return ans + 1;
    }
};
```

Logic of the Code

The main idea is:

Instead of actually removing people from the circle, we calculate where the winner will be.

1. Start with one person

With only one person, that person is obviously the winner.

Using **0-based indexing**:

```
Person: 0
Winner: 0
```

Therefore:

```
int ans = 0;
```

2. Add people one by one

Now suppose we already know the winner for `i - 1` people.

When we increase the circle to `i` people, the winner's position shifts by `k`.

So:

```
ans + k
```

But there are only `i` positions: `0` to `i-1`.

Therefore, we use `% i` to wrap around the circle:

```
ans = (ans + k) % i;
```

Why `% i`?

Because the game is **circular**.

For example, with 5 people:

```
0 1 2 3 4
```

If we move beyond 4, we come back to 0.

For example:

```
4 + 2 = 6
6 % 5 = 1
```

So:

```
4 → 0 → 1
```

The modulo operation gives us the correct circular position.

Example: `n = 5, k = 2`

We start:

```
ans = 0
```

```
i = 2
```

```
ans = (0 + 2) % 2;
      = 0;
```

Winner index:

```
0
```

```
i = 3
```

```
ans = (0 + 2) % 3;
      = 2;
```

Winner index:

```
2
```

```
i = 4
```

```
ans = (2 + 2) % 4;
      = 0;
```

Winner index:

```
0
```

```
i = 5
```

```
ans = (0 + 2) % 5;
      = 2;
```

Winner index:

```
2
```

But the actual people are numbered:

```
1 2 3 4 5
```

Since our answer is 0-based:

```
index 0 → person 1
index 1 → person 2
index 2 → person 3
```

Therefore:

```
return ans + 1;
```

Answer:

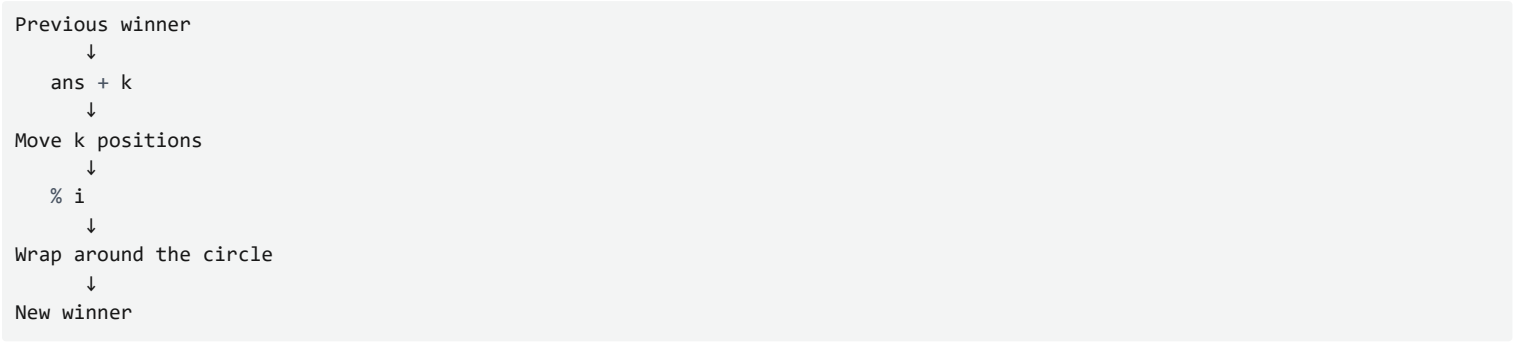
```
3
```

The Important Formula

The entire logic comes down to:

```
ans = (ans + k) % i;
```

Meaning:



Why this is efficient

We don't actually create the circle or erase elements.

We only calculate the winner's position.

- **Time:** $O(n)$
- **Space:** $O(1)$

One-line intuition

Start with winner 0 , then for every circle size i , shift the previous winner by k positions and use modulo i to handle the circular wrap-around.

Absolutely. The **most important thing** is to understand why this line works:

```
return (solve(n - 1, k) + k) % n;
```

Don't think of it as a magic formula. It comes from **reducing the problem from n people to $n-1$ people**.

Core Logic

Suppose we have n people:

```
0 1 2 3 4 ... n-1
```

and every k -th person is removed.

After removing one person, there are only:

```
n - 1
```

people left.

So instead of solving the whole problem for n people, we ask:

If I already know the winner among $n-1$ people, where will that winner be in the original n -person circle?

That's exactly what recursion does.

Step 1 – Solve the smaller problem

```
solve(n - 1, k)
```

This tells us:

"What is the winner's position when there are $n-1$ people?"

Suppose it returns:

2

That means:

```
Winner = index 2
```

inside the smaller circle.

Step 2 — The circle has shifted

This is the key idea.

When one person is eliminated from the original circle, the next person becomes the new starting point.

For example:

```
0 1 2 3 4
```

Suppose 2 is eliminated.

The remaining circle is:

```
3 4 0 1
```

Now, for the smaller problem, we can rename these positions:

```
3 → 0
4 → 1
0 → 2
1 → 3
```

So the smaller problem looks like:

```
0 1 2 3
```

We solve this smaller circle and get some winner index.

But that index belongs to the **renumbered smaller circle**, not the original circle.

Therefore, we need to shift the index back.

Step 3 — Shift by k

The position of the eliminated person determines where the new circle starts.

Because the elimination happens after moving k positions, the relationship between the smaller circle and the original circle is a shift of k .

Therefore:

```
solve(n - 1, k) + k
```

gives the corresponding position in the original circle.

Step 4 — Why $\% n$?

The original circle has n positions:

```
0 1 2 3 ... n-1
```

If:

```
solve(n - 1, k) + k
```

goes beyond $n-1$, we need to come back to the beginning.

That's exactly what modulo does:

```
(solve(n - 1, k) + k) % n
```

So $\% n$ means:

Keep the position inside the circular range 0 to $n-1$.

Therefore

The complete recurrence is:

```
solve(n, k) = (solve(n - 1, k) + k) % n;
```

It means:

```
Solve n-1 people
↓
Get their winner index
↓
Shift that index by k
↓
Use % n to wrap around the circle
↓
Get winner index for n people
```

Now understand the recursion

```
int solve(int n, int k) {

    if (n == 1)
        return 0;

    return (solve(n - 1, k) + k) % n;
}
```

Base case

```
if (n == 1)
    return 0;
```

One person means:

```
0
↑
only person
```

So winner = 0.

Recursive case

```
solve(n - 1, k)
```

First solves the smaller problem.

Then:

```
+ k
```

shifts the winner back to the original circle.

Then:

```
% n
```

handles circular wrapping.

Example: `n = 5, k = 2`

The recursion first goes down:

```
solve(5,2)
↓
solve(4,2)
↓
solve(3,2)
↓
solve(2,2)
↓
solve(1,2)
```

At the bottom:

```
solve(1,2) = 0
```

Now it starts returning.

For 2 people

```
(0 + 2) % 2  
= 0
```

So:

```
solve(2) = 0
```

For 3 people

```
(0 + 2) % 3  
= 2
```

So:

```
solve(3) = 2
```

For 4 people

```
(2 + 2) % 4  
= 0
```

So:

```
solve(4) = 0
```

For 5 people

```
(0 + 2) % 5  
= 2
```

So:

```
solve(5) = 2
```

Finally:

```
return solve(n, k) + 1;
```

because we calculated using 0-based indexing.

```
index 2 → person 3
```

Therefore:

```
Answer = 3
```

The one thing to remember

Don't try to memorize:

```
(solve(n - 1, k) + k) % n
```

as a random formula.

Remember the **three operations**:

```
solve(n - 1, k)  
  ↓  
smaller problem  
  ↓  
  + k  
  ↓  
shift back to original circle  
  ↓  
  % n  
  ↓  
wrap around the circle
```

So the entire logic is:

Find the winner for $n-1$ people, shift its position by k because the starting point changes after an elimination, and use modulo n to keep the position inside the circular range.

That's the actual idea behind the recursive Josephus solution.